

ResNet Workload Hardware Optimization

EE6455 Fall 2025 - Gem5 Project Report

1. Introduction and Methodology

1.1 Project Overview

此次期末專題針對在 gem5 全系統模擬(FS)下執行的多執行緒 ResNet 之workload(resnet_mt)進行硬體架構配置，並透過調整多個參數來找出最佳配置。目標是在 1500-unit cost budget內，最小化simSeconds。

1.2 模擬設定

- **模擬模式**:全系統(FS)搭配 Ruby/CHI 一致性協議
- **指令集架構**:X86
- **CPU 頻率**:3 GHz(固定)
- **開機方式**:使用 KVM 快速開機進行 Linux 啟動, 然後切換至 O3 CPU 進行 ROI 模擬
- **workload**:resnet_mt --channels 16 --size 8 --blocks 4 --threads N
- **Cache階層**:三層 Ruby/CHI(private L1 → private L2 → shared L3)
- **網路拓撲**:Ring topology搭配 SimplePt2Pt

1.3 ROI 量測

統計數據將在 ResNet ROI 階段內收集。rcS 腳本在開機後觸發 m5 exit 以從 KVM 切換至 O3, 然後在workload前呼叫 resetstats, 完成後呼叫 dumpstats。最終 stats dump 的 simSeconds 值代表 ROI 執行時間。

2. 系統配置

2.1 架構概述

此次系統使用三層Cache, 每個核心具有private L1I/L1D 和 L2 Cache, 以及shared的 L3 Home Node。所有核心透過 Ring topology連接至記憶體控制器。

2.2 成本計算公式

CPU 成本(per core):

$$\text{score} = \alpha \times W + \beta \times (R/16) + \gamma \times (I/32) + \gamma \times (F/32) + \delta \times ((LQ+SQ)/64) + \epsilon \times (Wi+Wc)$$

其中 $W = (Wf+Wd+Wr+Wi+Ww+Wc)/6$, 且 $(\alpha, \beta, \gamma, \delta, \epsilon) = (10, 12, 8, 6, 2)$

$$C_LSQ = 0.05 \times \max(0, (LQ-32) + (SQ-32))$$

$$\text{per core成本} = 40 + \kappa \times \text{score} + C_BP + C_LSQ, \text{ 其中 } \kappa = 0.01$$

Cache成本:

$$L1 = 3 \times (\text{size}/16\text{KB}) \times \text{cores} \times 2 (I+D)$$

$$L2 = (\text{size}/16\text{KB}) \times \text{cores}$$

$$L3 = (\text{size}/64\text{KB}) \times \text{slices}$$

記憶體成本(DDR4):

$$C_mem = 1.3 \times [(60 \times \text{channels}) + (5 \times \text{freq_step} \times \text{channels}) + (10 \times \text{capacity_GB})]$$

NoC 成本(Ring):

$$C_NoC = 10 \times N_routers + 5 \times N_routers, \text{ 其中 } N_routers = \text{cores} \times 3 + 1 + L3_slices$$

2.3 配置比較

參數	基準 (run1)	最佳配置 (run_opt4)
核心數	4	4
寬度 (Wf/Wd/Wr/Wi/Ww/Wc)	12/10/10/12/10/10	12/12/12/12/12/12
ROB	50	192
LQ / SQ	128 / 128	192 / 192
Int/FP Regs	50 / 50	96 / 96

參數	基準 (run1)	最佳配置 (run_opt4)
L1 (per core)	32 KB	32 KB
L2 (per core)	256 KB	2048 KB
L3 (shared)	2048 KB	6144 KB
Memory	DDR4-2400 3GB	DDR4-2400 3GB
Total Cost	692	1236
simSeconds	0.001955	0.001474

加速比: $1.33 \times (0.001955 / 0.001474)$

2.4 成本明細(最佳化配置)

元件	計算	成本(單位)
CPU(4 核心)	4×59.96	239.84
L1 Cache	$3 \times (32/16) \times 4 \times 2$	48
L2 Cache	$(2048/16) \times 4$	512
L3 Cache	$(6144/64) \times 1$	96
記憶體(DDR4-2400)	$1.3 \times (60+10+30)$	130
NoC(Ring, 14 routers)	15×14	210
Total		1235.84 $\approx$ 1236

3. 實驗結果

3.1 完整實驗數據

我們進行了 35 組實驗，調整 CPU 參數、Cache 大小和配置。圖 1 展示了所有實驗配置的 Design Space Exploration 結果，橫軸為總成本，縱軸為執行時間 (simSeconds)。

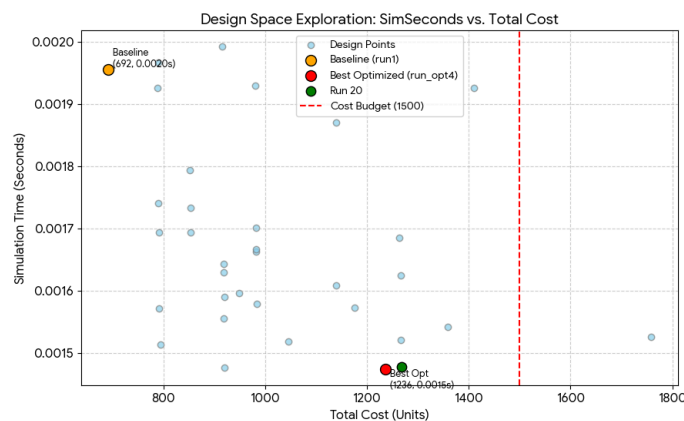


圖 1: Design Space Exploration (SimSeconds vs. Total Cost)。紅色虛線代表 1500 單位的成本預算上限。

Run	simSeconds	成本	ROB	LQ/SQ	L2 (KB)	L3 (KB)	暫存器
run_opt4	0.001474	1236	192	192/192	2048	6144	96/96
run18	0.001476	920	128	128/128	1024	4096	96/96
run20	0.001478	1268	192	192/192	2048	8192	96/96
run8	0.001513	794	192	128/128	512	4096	96/96
run24	0.001518	1046	96	128/128	1536	4096	80/80
run1	0.001955	692	50	128/128	256	2048	50/50

註:綠色 = 最佳效能(run_opt4), 黃色 = 基準(run1)

3.2 實驗最佳結果

- **最佳配置(run_opt4)**:simSeconds = 0.001474, 成本 = 1236(<1500)
- **基準(run1)**:simSeconds = 0.001955, 成本 = 692
- **效能提升**:1.33× 加速比, 成本增加 79%
- **成本效率**:run18 以 920 成本達到 0.001476s(極佳性價比)

4. 分析與討論

4.1 L2/L3 Cache大小的影響

Cache大小對效能有顯著影響。比較具有相似 CPU 參數的配置：

- **L2 影響**:將 L2 從 1024KB 增加到 2048KB(run18 vs run_opt4)可改善效能
- **L3 影響**:將 L3 從 4096KB 增加到 8192KB 顯示效能增益有所降低
- **workload**:ResNet workload(16 通道 $\times$ $8 \times 8 \times 4$ 區塊)在較大的 L2 Cache中能良好容納處理, 減少 L3 和記憶體存取

4.2 LQ/SQ 大小的影響

Load Queue(LQ)和 Store Queue(SQ)大小顯著影響記憶體密集型workload：

- **大型 LQ/SQ(192/192)**:run_opt4 達到最佳效能, 允許更多未完成的記憶體操作
- **較小 LQ/SQ(128/128)**:run18 仍表現良好, 顯示適當的 ROB 和Cache可以補償
- **平衡方法**:run8 以 LQ=128/SQ=128 在較低成本下顯示良好效能

4.3 ROB 大小分析

ROB 大小顯示較為有趣的結果：

- **ROB=192(最佳)**:大多數頂級效能配置使用 ROB=192
- **ROB=50(基準)**:但當Cache階層是瓶頸時, 較大的 ROB 沒有幫助
- **結論**:對於此workload, 投資在Cache容量比 ROB 大小提供更好的回報

4.4 瓶頸分析

由於卷積運算且workload主要是記憶體密集型：

- **記憶體存取模式**: 3×3 卷積需要跨特徵圖的跨步存取
- **資料重用**:較大的 L2 Cache改善權重矩陣的資料重用
- **執行緒競爭**:4 核心配置搭配私有 L2 最小化Cache一致性開銷
- **記憶體頻寬與延遲**:儘管是記憶體密集型應用, 實驗比較 (run20 vs run20_dual) 顯示dual channel因引入額外存取延遲, 效能反而下降 (0.001542s vs 0.001478s)。這證實 2MB L2 Cache 有效降低了系統對Main Memory頻寬的依賴, 使效能瓶頸轉為記憶體存取延遲 (Latency-bound), 因此單通道配置更具優勢。

4.5 Cost-Performance Trade-offs

- **高效率**:run18(成本=920, 0.001476s)- 每單位成本最佳效能
- **最大效能**:run_opt4(成本=1236, 0.001474s)- 預算內的絕對最佳
- **報酬遞減**:超過 1000 成本單位後, 效能增益變得微小

5. 結論

本專案展示了 gem5 上 ResNet 推論的硬體優化。主要結論：

- **最佳配置**:run_opt4 達到 0.001474s simSeconds, 成本 1236(相對基準加速 1.33×)
- **主要瓶頸**:記憶體階層(L2/L3 Cache)主導效能, 而非 CPU execution width
- **設計策略**:對於memory-bound convolution workload, 投資較大的 L2 Cache和 LQ/SQ 大小
- **Cost efficiency**:平衡配置(run18)提供極佳的效能成本比

6. 分工表

組員	任務與貢獻	貢獻度 (%)
鄭凱權 / 114064521	系統配置、資料收集、成本計算、實驗執行	50%
滕彥宜 / 114061598	系統配置、資料收集、結果分析、報告撰寫	50%